



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09-02

NOMBRE COMPLETO: Castillo Martínez Diego Leonardo

N° de Cuenta: 319041538

GRUPO DE LABORATORIO: 11

GRUPO DE TEORÍA: 06

SEMESTRE 2024-2

FECHA DE ENTREGA LÍMITE: 23 de octubre de 2024

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHG MONOPOLY. animado desplazándose las letras de izquierda a derecha como si fuera letrero LCD/LED de forma cíclica

Para este ejercicio creé un alfabeto con la palabra mencionada, con mis letras de mi abecedario, siendo de vital importancia que estuviera escalado en 512x512



Posteriormente, Se generaron 8 estados, mas un estado que simboliza la textura vacía o bien transparente, se opto por abordar el problema por medio de una maquina de estados, en donde se va recorriendo letra a letra y cada letra simboliza un nuevo estado.

```

// van a existir 8 estados y una base    --- la podriamos considerar como un apoyo
// Nulo
float toffsetLetreroNulou = 0.0f;
float toffsetLetreroNulov = 0.0f;
// base
float toffsetLetrerobaseu = 0.0f;
float toffsetLetrerobasev = 0.0f;
// primer estado
float toffsetLetrero1u = 0.0f;
float toffsetLetrero1v = 0.0f;
// Segundo estado
float toffsetLetrero2u = 0.0f;
float toffsetLetrero2v = 0.0f;
// Tercer estado
float toffsetLetrero3u = 0.0f;
float toffsetLetrero3v = 0.0f;
// Cuarto estado
float toffsetLetrero4u = 0.0f;
float toffsetLetrero4v = 0.0f;
// Quinto estado
float toffsetLetrero5u = 0.0f;
float toffsetLetrero5v = 0.0f;
// Sexto estado
float toffsetLetrero6u = 0.0f;
float toffsetLetrero6v = 0.0f;
// Septimo estado
float toffsetLetrero7u = 0.0f;
float toffsetLetrero7v = 0.0f;
// Octavo estado

```

```

// Octavo estado
float Auxiliaru = 0.0f;
float Auxiliarv = 0.0f;
Window mainWindow;
std::vector<Mesh*> meshList;
std::vector<Shader> shaderList;

```

Declaramos la textura de nuestro archivo con nuestra palabra, cargando nuestra textura

```

Texture brickTexture;
Texture dirtTexture;
Texture plainTexture;
Texture pisoTexture;
Texture AgaveTexture;
Texture FlechaTexture;
Texture NumerosTexture;
Texture Numero1Texture;
Texture Numero2Texture;
Texture LetreroBase;
Texture Letrero;

```

```

NumerosTexture.LoadTextureA();
Numero1Texture = Texture("Textures/numero1.tga");
Numero1Texture.LoadTextureA();
Numero2Texture = Texture("Textures/numero2.tga");
Numero2Texture.LoadTextureA();
LetreroBase = Texture("Textures/LetreroFinal.tga");
LetreroBase.LoadTextureA();
Letrero = Texture("Textures/Letrero.tga");
Letrero.LoadTextureA();

```

Es importante mencionar que creamos un nuevo objeto, para que exista soporte al momento de ir mostrando 1 a una las 8 letras de nuestro plano.

```

unsigned int LetreroIndices[] = {
    0, 1, 2,
    0, 2, 3,
};

GLfloat LetreroVertices[] = {
    // definimos 3 columnas y vamos de 1 en uno para cada vertice
    -.5f, 0.0f, 0.5f,      0.0f, 0.844f,      0.0f, -1.0f, 0.0f,
    .5f, 0.0f, 0.5f,      .125f, 0.844f,      0.0f, -1.0f, 0.0f,
    .5f, 0.0f, -0.5f,     .125f, 1.0f,         0.0f, -1.0f, 0.0f,
    -.5f, 0.0f, -0.5f,    0.0f, 1.0f,         0.0f, -1.0f, 0.0f,
};

//INDICE 0
Mesh *obj1 = new Mesh();
obj1->CreateMesh(vertices, indices, 32, 12);
meshList.push_back(obj1);
//1
Mesh *obj2 = new Mesh();
obj2->CreateMesh(vertices, indices, 32, 12);
meshList.push_back(obj2);
//2
Mesh *obj3 = new Mesh();
obj3->CreateMesh(floorVertices, floorIndices, 32, 6);
meshList.push_back(obj3);

```

```

//3
Mesh* obj4 = new Mesh();
obj4->CreateMesh(vegetacionVertices, vegetacionIndices, 64, 12);
meshList.push_back(obj4);
//4
Mesh* obj5 = new Mesh();
obj5->CreateMesh(flechaVertices, flechaIndices, 32, 6);
meshList.push_back(obj5);
//5
Mesh* obj6 = new Mesh();
obj6->CreateMesh(scoreVertices, scoreIndices, 32, 6);
meshList.push_back(obj6);
//6
Mesh* obj7 = new Mesh();
obj7->CreateMesh(numeroVertices, numeroIndices, 32, 6);
meshList.push_back(obj7);

Mesh* obj8 = new Mesh();
obj8->CreateMesh(LetreroVertices, LetreroIndices, 32, 6);
meshList.push_back(obj8);

```

Un aspecto importante para destacar es que nos basamos en la malla (mesh) utilizada para renderizar cada columna de la textura de números. En la definición de las coordenadas u y v , determinamos que en el eje u se utilizará un octavo ($1/8$) de la textura, y en el eje v definimos 80 píxeles, lo que corresponde a la altura de cada fila y columna en nuestra textura. Esto nos permite seccionar la textura de manera precisa.

Posteriormente, implementamos una máquina de estados para controlar la animación del cartel. Dividimos el cartel en 8 secciones horizontales, y asignamos una altura específica a cada columna. El cartel consta de 3 columnas, cada una representando una palabra distinta, que a su vez se divide en 8 partes iguales. Cada una de estas partes corresponde a un estado. El comportamiento de la máquina de estados se basa en un ciclo: el estado 1 pasa al estado 2, el estado 2 al estado 3, y así sucesivamente hasta llegar al estado 8. Cuando el ciclo se completa, regresa al estado base y comienza de nuevo.

Conforme avanza el tiempo, se realiza un desplazamiento en el eje x de la textura, restando a la coordenada u , lo que provoca que la animación comience desde la última letra de cada palabra y avance hacia la primera. Esta técnica nos permite crear un efecto visual en el que las letras se desplazan de izquierda a derecha, dando lugar a una animación fluida que simula el arrastre de los estados de una columna a la siguiente.

```

if (PasoSegundo) {
    if (toffsetLetrerobaseu < 0.0 && faseActual==1) {
        toffsetLetrerobaseu = 7 * .125;
        toffsetLetrerobasev = 2 * -.1562;
    }
    else if (toffsetLetrerobaseu < 0.0 && faseActual == 2) {
        toffsetLetrerobaseu = 7 * .125;
        toffsetLetrerobasev = -.1562;
    }
    else if (toffsetLetrerobaseu < 0.0 && faseActual == 0) {
        toffsetLetrerobaseu = 7 * .125;
        toffsetLetrerobasev = 0.0;
    }

    // Guardar la última letra
    float Auxiliaru = toffsetLetrero8u;
    float Auxiliarv = toffsetLetrero8v;

    // Desplazar todas las letras hacia la derecha
    toffsetLetrero8u = toffsetLetrero7u;
    toffsetLetrero8v = toffsetLetrero7v;

    toffsetLetrero7u = toffsetLetrero6u;
    toffsetLetrero7v = toffsetLetrero6v;

    toffsetLetrero6u = toffsetLetrero5u;
    toffsetLetrero6v = toffsetLetrero5v;

```

```

    toffsetLetrero5u = toffsetLetrero4u;
    toffsetLetrero5v = toffsetLetrero4v;

    toffsetLetrero4u = toffsetLetrero3u;
    toffsetLetrero4v = toffsetLetrero3v;

    toffsetLetrero3u = toffsetLetrero2u;
    toffsetLetrero3v = toffsetLetrero2v;

    toffsetLetrero2u = toffsetLetrero1u;
    toffsetLetrero2v = toffsetLetrero1v;

    // Asignar el valor de la base de la textura a la primera casilla
    toffsetLetrero1u = toffsetLetrerobaseu;
    toffsetLetrero1v = toffsetLetrerobasev;

    // Actualizar la base de la textura para la siguiente letra
    toffsetLetrerobaseu -= .125;
}

// primera letra
toffset = glm::vec2( toffsetLetrero1u, toffsetLetrero1v);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(3.0f - (7 * 1.0), 11.5f + movCartel, 1.1f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LetreroBase.UseTexture();

```

```

color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LetreroBase.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();
//Segunda letra
toffset = glm::vec2(toffsetLetrero2u, toffsetLetrero2v);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(3.0f - (6 * 1.0), 11.5f + movCartel, 1.1f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LetreroBase.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();
//Tercera Letra
toffset = glm::vec2(toffsetLetrero3u, toffsetLetrero3v);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(3.0f - (5 * 1.0), 11.5f + movCartel, 1.1f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LetreroBase.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();
// Cuarta letra

```

Nota: para evitar el exceso de capturas, se tomo en cuenta solamente las tres primeras letras pues en realidad el instanciarlas sigue el mismo patrón

Para finalizar, simplemente instanciamos nuestro arco, con cada parte por separado.

```

// Instancia del puente
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0, -.9f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Arco.RenderModel();
// Instancia de nuestro cartel
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0, 11.0f + movCartel, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Cartel.RenderModel();
glDisable(GL_BLEND);

```


2.- Separar las cabezas del Dragón y agregar las siguientes animaciones:

- Movimiento del cuerpo ida y vuelta.
- Aleteo
- Cada cabeza se mueve de forma diferente de acuerdo a una función/algorithmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc..)
- Cada cabeza debe de verse de un color diferente: roja, azul, verde, blanco y café.

El primer paso fue realizar la separación de cabezas del dragón, asignándole un color diferente a cada cabeza.

Después, introducimos cada una de las piezas dentro de nuestro código, cargando los archivos y colocándolos, con ayuda de la jerarquía, en su posición correcta:

```
Model ala_derecha;  
Model ala_izquierda;  
Model cuerpo_dragon;  
Model cabeza1;  
Model cabeza2;  
Model cabeza3;  
Model cabeza4;  
Model cabeza5;
```

```
ala_derecha = Model();  
ala_derecha.LoadModel("Models/ala_derecha.obj");  
ala_izquierda = Model();  
ala_izquierda.LoadModel("Models/ala_izquierda.obj");  
cuerpo_dragon = Model();  
cuerpo_dragon.LoadModel("Models/cuerpo_dragon.obj");  
cabeza1 = Model();  
cabeza1.LoadModel("Models/cabeza1.obj");  
cabeza2 = Model();  
cabeza2.LoadModel("Models/cabeza2.obj");  
cabeza3 = Model();  
cabeza3.LoadModel("Models/cabeza3.obj");  
cabeza4 = Model();  
cabeza4.LoadModel("Models/cabeza4.obj");  
cabeza5 = Model();  
cabeza5.LoadModel("Models/cabeza5.obj");
```

Comenzamos con el cuerpo, y decidimos que avanzara y girara, manteniéndolo todo dentro del área del piso. Para ello, declaramos variables que controlan su posición, la rotación y la velocidad.


```
float dragonPosX = 0.0f; //Posición inicial
float velocidadDragon = 0.5f;
bool xPositivo = false; //Bandera de dirección del movimiento
float dragonRotationY;
```

```
if (xPositivo) {
    dragonPosX += velocidadDragon * deltaTime;
    if (dragonPosX >= 270.0f) {
        xPositivo = false; //Cambiar dirección
    }
}
else {
    dragonPosX -= velocidadDragon * deltaTime;
    if (dragonPosX <= -270.0f) {
        xPositivo = true; //Cambiar dirección
    }
}
//Rotando
if (xPositivo) {
    dragonRotationY = 180.0f;
}
else {
    dragonRotationY = 0.0f;
}
```

```
//Renderizar cuerpo
model = glm::mat4(1.0);
//Multiplicar por valor mayor, longitud m+as alta y periodo se acorta
/*model = glm::translate(model, glm::vec3(0.0f-angulovaria/6, 5.0f+3*sin(glm::radians(angulovaria*8)),
6.0));*/
model = glm::translate(model, glm::vec3(dragonPosX, 5.0f, 6.0f)); // La posición en X es la que se modifica
model = glm::rotate(model, glm::radians(dragonRotationY), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotar el dragón
modelaux = model;
//model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
//model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
//Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cuerpo_dragon.RenderModel();
```

Luego, para las alas, creamos variables que regulan la velocidad del aleteo, su ángulo y su posición actual. Estas nos permiten subir y bajar el ala según si ha alcanzado el límite de ángulo permitido, evitando que haga una rotación completa sobre el eje X.

```
float anguloAla = 35.0f; //Inicio de angulo
float velocidadAla = 2.0f;
bool alaArriba = false; //Bandera que comienza con ala arriba
```

```

if (alaArriba) {
    anguloAla += velocidadAla * deltaTime; //Subimos ala
    if (anguloAla >= 35.0f) {
        alaArriba = false; //Cambiar la dirección para bajar
    }
}
else {
    anguloAla -= velocidadAla * deltaTime; //Bajamos ala
    if (anguloAla <= -75.0f) {
        alaArriba = true; //Cambiar la dirección para subir
    }
}
}

```

```

//-----Ala derecha
model = modelaux;
model = glm::translate(model, glm::vec3(0.4f, 0.64f, -0.48f));
model = glm::rotate(model, glm::radians(anguloAla), glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
ala_derecha.RenderModel();
//-----Ala izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(0.38f, 0.679f, 0.439f));
model = glm::rotate(model, glm::radians(-anguloAla), glm::vec3(1.0f, 0.0f, 0.0f)); // Invertir el ángulo
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
ala_izquierda.RenderModel();

```

Para la primera cabeza, se implementa un movimiento senoidal en el eje Y, utilizando la función `sin()` para que la posición oscile con el tiempo. Esto hace que la cabeza se desplace hacia arriba y hacia abajo siguiendo un patrón de onda senoidal, con una amplitud de 2 unidades. La velocidad de la oscilación está determinada por el valor de `angulovaria` multiplicado por 4.

```

//-----Cabeza 1 (Movimiento senoidal)
model = modelaux;
//model = glm::translate(model, glm::vec3(-1.09f, 1.3f, 0.73f));
model = glm::translate(model, glm::vec3(-1.09f, 1.3f + 2.0f * sin(glm::radians(angulovaria * 4)), 0.73f)); // Movimiento senoidal en Y
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza1.RenderModel();

```

Para la segunda cabeza, se implementa un movimiento circular utilizando las funciones `cos()` y `sin()` para que la cabeza trace un círculo con un radio de 2 unidades. El ángulo de rotación está controlado por el valor de `angulovaria`, y al multiplicarlo por 2, se ajusta la velocidad del movimiento. La posición en el eje Y se mantiene constante en 0.25f.

```

//-----Cabeza 2 (Movimiento circular)
float radio = 2.0f; // Radio del movimiento circular
model = modelaux;
//model = glm::translate(model, glm::vec3(-1.49f, 0.25f, 0.9f));
model = glm::translate(model, glm::vec3(-1.49f + radio * cos(glm::radians(angulovaria * 2)), 0.25f, 0.9f + radio * sin(glm::radians(angulovaria * 2)))); // Movimiento circular en el plano XZ
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza2.RenderModel();

```

Para la tercera cabeza, se aplica un movimiento en forma de lemniscata (o símbolo de "infinito"), utilizando las variables xLemniscata y yLemniscata, que se calculan con funciones trigonométricas en función de angulovaria. Este cálculo crea un patrón de trayectoria con forma de "infinito". La ecuación está ajustada para mantener la forma característica de la lemniscata de Bernoulli. Como resultado, la cabeza sigue un recorrido de doble bucle alrededor de un punto fijo, mientras la posición en el eje Y permanece constante.

```
//-----Cabeza 3 (Roja) (Movimiento en lemniscata)
float xLemniscata = (2.0f * cos(glm::radians(angulovaria * 4))) / (1 + sin(glm::radians(angulovaria * 4))) *
sin(glm::radians(angulovaria * 4));
float yLemniscata = (2.0f * sin(glm::radians(angulovaria * 4)) * cos(glm::radians(angulovaria * 4))) / (1 +
sin(glm::radians(angulovaria * 4)) * sin(glm::radians(angulovaria * 4)));
model = modelaux;
//model = glm::translate(model, glm::vec3(-1.0f, 1.35f, -0.5f));
model = glm::translate(model, glm::vec3(-1.0f + xLemniscata, 1.35f, -0.5f + yLemniscata));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza3.RenderModel();
```

Para la cuarta cabeza, se implementa un movimiento pendular exclusivamente en el eje X. Este movimiento se genera con la función sin(), imitando el comportamiento de un péndulo. La variable amplitud Péndulo regula la magnitud del desplazamiento, es decir, cuánto se mueve la cabeza de un lado a otro, mientras que velocidad Péndulo determina la rapidez del balanceo.

```
//-----Cabeza 4 (Verde) (Movimiento pendular)
float amplitudPendulo = 1.5f; // Amplitud del movimiento pendular
float velocidadPendulo = 2.0f; // Velocidad del péndulo
model = modelaux;
//model = glm::translate(model, glm::vec3(-1.5f, 0.25f, -0.8f));
model = glm::translate(model, glm::vec3(-1.5f + amplitudPendulo * sin(glm::radians(angulovaria *
velocidadPendulo)), 0.25f, -0.8f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza4.RenderModel();
```

Para la quinta y última cabeza, se implementa un movimiento en espiral de Arquímedes. El radio de la espiral, representado por r, se calcula según una fórmula en el código, donde el parámetro a establece el tamaño inicial del círculo y b regula la velocidad a la que la espiral se expande conforme aumenta el valor de Angulo varia. La posición de la cabeza se actualiza mediante las funciones cos() y sin() para generar un movimiento circular que se expande progresivamente en el plano.

```
//-----Cabeza 5 (Movimiento espiral de arquímedes)
float a = 0.2f; // Tamaño inicial del círculo
float b = 0.05f; // Controla qué tan rápido se expande la espiral
float r = a + b * angulovaria; // Radio que varía lentamente con angulovaria
model = modelaux;
model = glm::translate(model, glm::vec3(-1.6f, 0.55f, 0.11f));
model = glm::translate(model, glm::vec3(-1.6f + r * cos(glm::radians(angulovaria * 2)), 0.55f, 0.11f + r *
sin(glm::radians(angulovaria * 2)))); // Espiral en el plano XZ
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza5.RenderModel();
```

La configuración de la cámara y nuestro deltaTime son diferentes al anterior ejercicio

```
GLfloat now = glfwGetTime();
deltaTime = now - lastTime;
deltaTime += (now - lastTime) / limitFPS;
lastTime = now;

angulovaria += 0.5f*deltaTime;
```

```
//Recibir eventos del usuario
glfwPollEvents();
camera.keyControl(mainWindow.getKeys(), deltaTime);
camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
```

1. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El ejercicio 1 presentó varios desafíos inesperados. Inicialmente, intenté realizar la animación utilizando como referencia el ejemplo de la práctica, específicamente el movimiento de la textura de la flecha. Sin embargo, a pesar de los numerosos intentos, no logré obtener un patrón coherente; parecía que la textura se "glitcheaba" o mostraba comportamientos erráticos. Después de dedicar mucho tiempo tratando de solucionar el problema sin éxito, decidí cambiar de enfoque.

Tras una cuidadosa reflexión, opté por implementar una máquina de estados para manejar la animación. Este cambio resultó ser mucho más efectivo y permitió un mayor control sobre el flujo de los estados, logrando finalmente el comportamiento deseado en la animación.

2. Conclusión:

Esta práctica fue tanto desafiante como enriquecedora. Al principio, pensé que sería relativamente sencilla, pero después de invertir más de 8 horas en el ejercicio 1 sin éxito, me di cuenta rápidamente de la complejidad que implicaba. A pesar de las dificultades, fue una experiencia muy entretenida, ya que me obligó a abordar el problema desde una perspectiva completamente diferente, algo que no hacía desde hace tiempo.

De hecho, tuve que recordar un ejemplo de VHDL de mis prácticas de diseño digital para poder aplicar una solución basada en una máquina de estados. Este cambio de enfoque, junto con una comprensión profunda del código, me permitió resolver finalmente el ejercicio y comprender de manera adecuada la lógica detrás de la animación de texturas. Este

proceso me reafirmó la importancia de tener flexibilidad mental para abordar problemas complejos desde distintos ángulos.

3. Bibliografía

Electrónica FP. (2023, 26 abril). *Máquinas de estado. Ejemplos* [Vídeo]. YouTube.

Recuperado el 22 de octubre de 2024 de

<https://www.youtube.com/watch?v=NB5rjK1dQ9c>